

FloatChat — Frontend Engineering & Integration Specification

Document: Frontend Architecture, UX, API Integration & Implementation Instructions

Backend: Existing FloatChat Agentic AI Marine Intelligence Platform

Frontend: Separate repository

Primary objective: Build a production-grade conversational marine intelligence interface that integrates with the existing backend without coupling the UI to backend agent/MCP internals.

1. Objective

Build a standalone frontend for **FloatChat**, an Agentic AI-powered Marine Intelligence Platform.

The frontend must allow users to:

- Ask marine-related questions in natural language.
- Conduct multi-turn conversations.
- View AI-generated marine intelligence.
- Explore PFZs on an interactive map.
- View SST, chlorophyll, waves, currents, weather and other marine layers.
- View hazards and active restrictions.
- Receive safety alerts.
- Visualize routes and route alternatives.
- Explore charts and time-series data.
- Understand why the system produced a recommendation.
- Inspect evidence and source provenance.
- Work with Indian regional languages.
- Handle degraded, stale, unavailable, or incomplete data gracefully.
- Receive streaming updates while the backend agentic system executes.

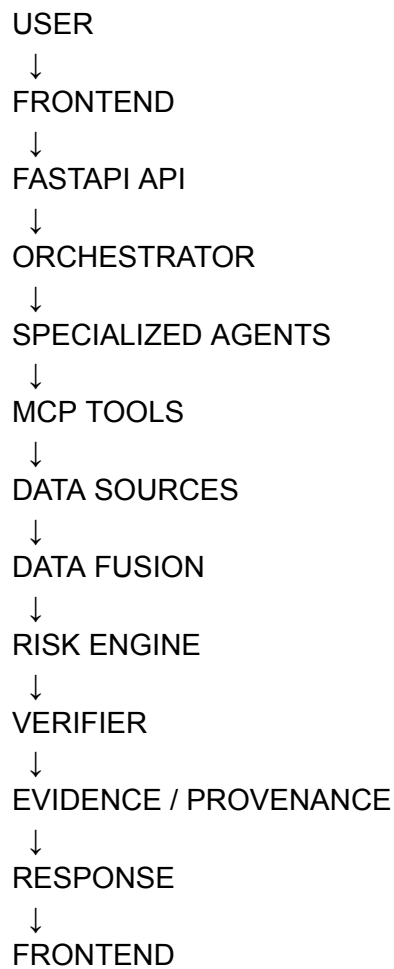
The frontend must **not implement marine intelligence itself**.

The frontend is a presentation and interaction layer over the backend.

2. Critical Architectural Rule

The backend is a black box.

The frontend must communicate only through the public API.



The frontend must **never directly access**:

- PostgreSQL
 - PostGIS
 - pgvector
 - MCP
 - INCOIS APIs
 - ISRO/MOSDAC APIs
 - IMD APIs
 - NHO/NAVTEX systems
 - internal ML models
 - internal agents
 - internal planner
 - internal executor
 - internal risk engine.
-

3. Recommended Technology Stack

Use:

Framework

Next.js + React + TypeScript

Use the App Router.

Styling

Use a consistent component system such as:

- Tailwind CSS
- shadcn/ui or equivalent component architecture

Do not create arbitrary one-off styling for every component.

State

Use a clear separation between:

Server/API State
+
Conversation State
+
UI State
+
Map State

Recommended:

- TanStack Query for API/server state.
- Zustand or React Context for UI/session state where required.

Do not put the entire application into one global store.

Mapping

Use a geospatial map framework such as:

- MapLibre GL JS
- Mapbox GL JS

Prefer GeoJSON-based rendering.

Charts

Use a production-quality chart library such as:

- Recharts
- Apache ECharts

API Client

Generate TypeScript types from the backend OpenAPI specification wherever possible.

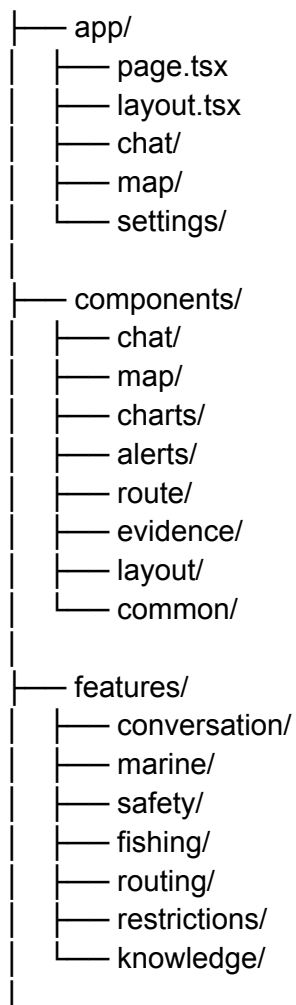
Do not manually duplicate backend schemas.

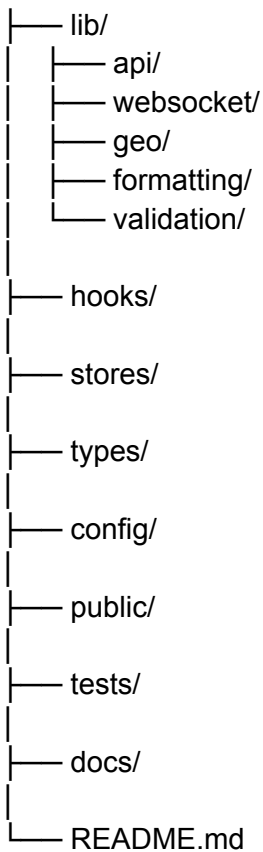
4. Repository Structure

Create a clean frontend repository.

Recommended:

floatchat-frontend/





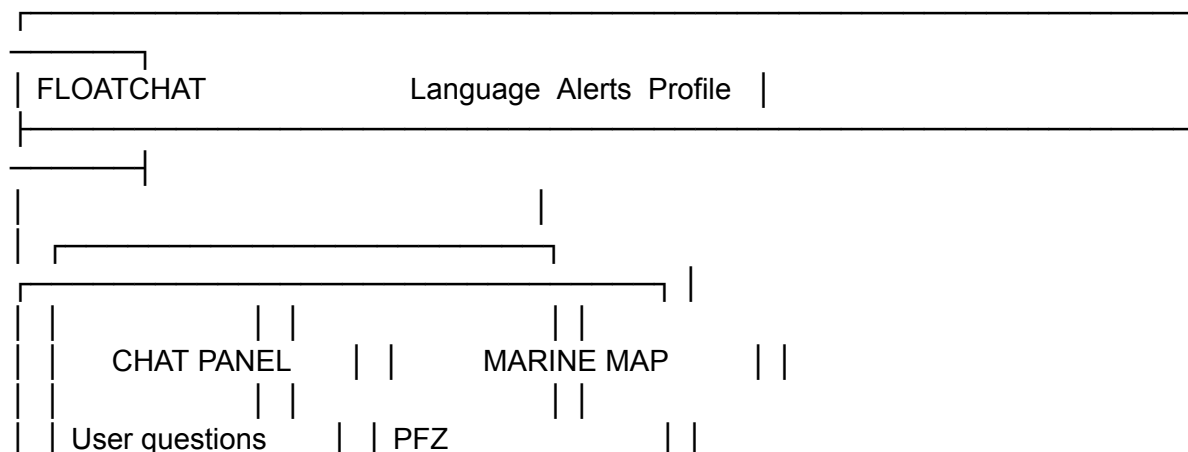
Keep domain-specific frontend logic inside `features/`.

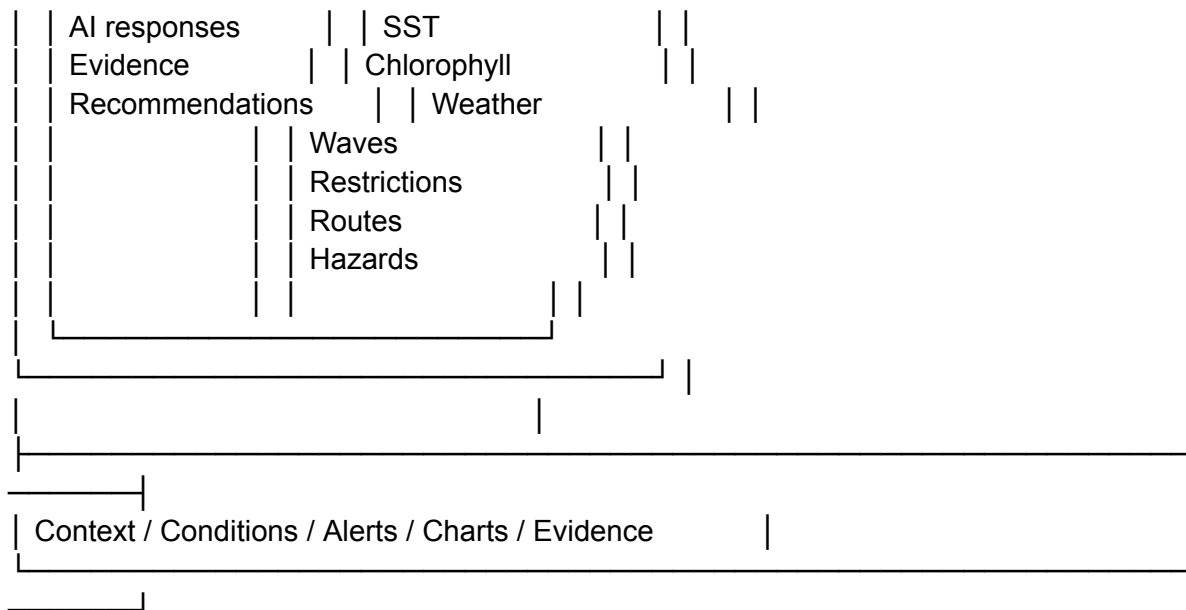
Do not place the entire application logic into `page.tsx`.

5. Primary Application Layout

The primary interface should be a marine intelligence workspace.

Recommended layout:





Desktop should support:

Chat + Map

as the primary workspace.

Mobile/tablet should collapse into:

Chat

Map

Insights

tabs or equivalent navigation.

6. Main Navigation

Provide:

Chat

Primary conversational interface.

Marine Map

Full-screen geospatial exploration.

Alerts

Active marine safety alerts and restrictions.

Insights

Charts, trends and marine analytics.

History

Previous conversations.

Settings

Language, units, location and notification preferences.

7. Chat Interface

The chat is the primary interaction mechanism.

The user should be able to type natural language such as:

Where is the nearest PFZ today?

Is it safe to go fishing tomorrow morning?

What are the wave conditions near Kochi?

Show areas with high chlorophyll and favourable SST.

Why has fish productivity declined here?

Find the safest route from Kochi to this fishing location.

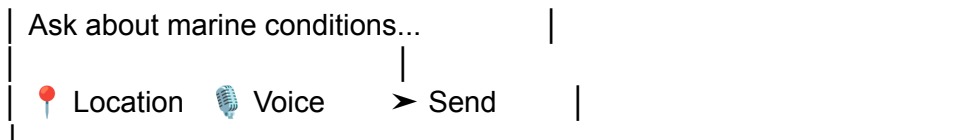
Do not force users to select an agent or dataset.

8. Chat Input

The input should support:

- text
- location attachment
- map-selected location
- voice input
- optional image/file attachment if later supported

Example:



The frontend sends natural language to:

POST /api/v1/orchestrate

9. Conversation State

Every conversation should have:

session_id

Every orchestration run should have:

run_id

request_id

Frontend should persist the `session_id`.

Do not persist backend agent/task state as frontend state.

10. Multi-Turn Conversation

The interface must support:

User:

What are the conditions near Kochi?

Assistant:

Current conditions are...

User:

What about tomorrow morning?

Assistant:

Tomorrow morning...

User:

Would it be safe for a small fishing boat?

Assistant:

...

The frontend should send the same `session_id`.

The backend determines contextual interpretation.

Do not implement your own semantic conversation resolution in the frontend.

11. Streaming Responses

The backend may stream orchestration progress.

The frontend must support events such as:

run.started
plan.created
task.started
task.completed
agent.started
agent.completed
tool.started
tool.completed
fusion.completed
risk.completed
verification.completed
response.ready
run.completed
run.failed

Example UI:

Analyzing your request...

- ✓ Understanding request
- ✓ Checking marine conditions
- ✓ Checking weather
- ✓ Checking restrictions
- Assessing safety
- Preparing recommendation

Do not expose internal prompts.

Do not expose raw model reasoning.

Do not display private chain-of-thought.

Display only safe execution status.

12. Agent Transparency

The frontend may communicate high-level progress:

- Checking weather conditions
- Checking marine conditions
- Checking active restrictions
- Assessing safety
- Verifying recommendation

Do NOT display:

Planner generated DAG:
Task 1 → Task 2 → Task 4

or internal MCP payloads.

The user needs understandable system status, not implementation details.

13. Response Card

Every substantial answer should support:

Marine Safety Assessment

CAUTION

Conditions are not ideal for small boats.

Why?

Elevated wave conditions

Stronger winds expected

Active advisory nearby

Confidence: High

[View on Map]

[View Evidence]

14. Safety Classification

Use the backend classification.

Supported states:

SAFE
CAUTION
HIGH_RISK
CRITICAL
RESTRICTED
UNKNOWN

Never calculate these classifications in the frontend.

The frontend only maps the backend state to a visual representation.

15. Safety UI

Safety responses should be visually prominent.

Example:

SAFE
Conditions currently suitable based on available data.

CAUTION
Some conditions require additional care.

HIGH RISK
Conditions present significant operational risk.

CRITICAL
Do not proceed without appropriate authorization/safety assessment.

RESTRICTED
Area is subject to an active restriction/geofence.

UNKNOWN
Insufficient reliable information to determine safety.

Do not transform UNKNOWN into SAFE.

16. Hard Restriction Handling

Restrictions have higher priority than general recommendations.

If backend returns:

RESTRICTED

the UI must prominently display:

RESTRICTED AREA

and show:

- restriction geometry
- restriction type
- validity
- source
- explanation

Never display:

Safe

over an active restriction.

17. Marine Map

The map is a core component.

It should support:

Layers

PFZ

SST

Chlorophyll

Waves

Currents

Weather

Lightning

Cyclones

Restrictions

Geofences

MPA

EEZ

IMBL
Routes
Vessel

Use a layer control.

Example:

MAP LAYERS

- ☒ PFZ
 - ☐ SST
 - ☐ Chlorophyll
 - ☐ Waves
 - ☐ Currents
 - ☐ Weather
 - ☐ Hazards
 - ☒ Restrictions
 - ☐ Protected Areas
 - ☐ Route
-

18. Map Interaction

Users should be able to:

- zoom
 - pan
 - click features
 - select location
 - select region
 - draw/choose route endpoints
 - inspect PFZs
 - inspect restrictions
 - inspect hazards
 - center map on current location
-

19. Map Feature Popup

Clicking a feature should show:

Potential Fishing Zone

Location:

9.932° N
76.267° E

SST:
28.4 °C

Chlorophyll:
0.91 mg/m³

Source:
INCOIS / ISRO

Observation:
Today, 06:00 UTC

Confidence:
High

[\[Ask about this area\]](#)

For restrictions:

ACTIVE RESTRICTION

Type:
Temporary Maritime Restriction

Status:
Active

Valid:
Today 10:00 – 18:00

Source:
Navigation Advisory

[\[View Evidence\]](#)

20. GeoJSON

Prefer backend-provided GeoJSON.

Frontend must render:

Feature
FeatureCollection

Point
LineString
Polygon
MultiPolygon

Do not recreate backend geometries manually.

21. PFZ Experience

A PFZ query should produce both:

Conversational answer

and:

Map visualization

Example:

Nearest PFZ:
18.6 km southwest of your location.

Estimated distance:
18.6 km

Supporting conditions:
SST: 28.2°C
Chlorophyll: High

Map:

User Location •
\
\
PFZ polygon

22. Weather Experience

Weather information should support:

- current weather
- forecast
- wind

- rainfall
- lightning
- cyclone
- visibility where available
- wave conditions
- marine warnings

Do not hard-code weather values.

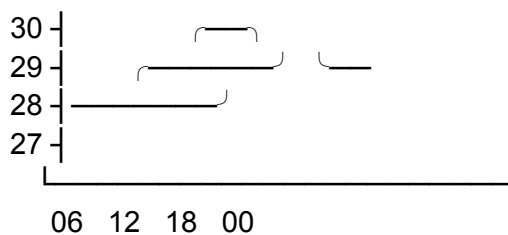
23. Time-Series Charts

Charts should support:

SST
Chlorophyll
Wave height
Wind
Current speed
Rainfall
Productivity
Risk

Example:

SST (°C)



Each chart should expose:

- unit
 - time
 - source
 - timestamp
 - freshness
 - confidence where available
-

24. Chart Interaction

Support:

- hover
- time selection
- zoom where useful
- source information
- current value
- historical comparison

Avoid excessive visual complexity.

25. Route Planning

The route interface should allow:

Origin

Destination

via:

- map selection
- current location
- search

Example:

Origin:

Kochi Harbour

Destination:

Fishing Zone A

[Find Safest Route]

Backend returns:

Selected route

Alternative routes

Distance

Risk

Restrictions

Blocking reasons

Geometry

26. Route Visualization

Display:

Selected Route
Alternative Route
Restricted Area
Hazard Area

The selected route should be clearly distinguishable from alternatives.

If route is blocked:

ROUTE BLOCKED

Reason:

Active maritime restriction intersects proposed route.

[View Restriction]

27. Alerts Center

Create a dedicated alerts interface.

Categories:

Cyclone
Lightning
High Waves
Strong Wind
Weather
Restriction
Geofence
Route
Data Quality

Each alert should include:

severity
title
message
location

validity
source

28. Alert Severity

Support:

INFO
ADVISORY
WARNING
HIGH
CRITICAL

The frontend must not reinterpret backend severity.

29. Active Restriction Interface

Provide a dedicated map/list for active restrictions.

Example:

ACTIVE MARITIME RESTRICTIONS

1. Temporary Navigation Restriction
Arabian Sea
Active until 18:00
2. Restricted Marine Zone
- ...

Clicking a restriction should zoom to its geometry.

30. Geofencing

When the backend reports that the user is:

approaching
inside
outside

a geofence, display an appropriate notification.

Example:

⚠ APPROACHING RESTRICTED AREA

You are approximately 1.8 km from
an active restricted zone.

[View on Map]

Do not independently calculate maritime legality.

31. Evidence Panel

Every important recommendation should support:

Why this answer?

Clicking it opens an evidence panel.

Example:

Evidence

INCOIS

Marine Forecast

Retrieved: 10:42 UTC

Valid: Today 12:00–18:00

Wave height:

2.1 m

IMD

Weather Forecast

Retrieved: 10:41 UTC

Wind:

24 km/h

Navigation Advisory

Active restriction:

Yes

32. Provenance

Show:

Source
Observed at
Retrieved at
Valid until
Freshness

Do not show database table names.

Do not show SQL queries.

33. Confidence

Display:

Confidence: High

with optional expandable explanation.

Possible values:

Very Low
Low
Medium
High
Very High

Do not calculate the confidence score in the frontend.

34. Data Freshness

Marine information can become stale.

The frontend should clearly distinguish:

LIVE
FRESH
RECENT
STALE

UNAVAILABLE

Example:

SST

Updated 22 min ago

- Fresh

or:

Marine Forecast

Updated 9 hours ago

⚠ Stale

35. Degraded Mode

If some data sources fail, the UI should not crash.

Example:

Limited information available

IMD weather data: Available

INCOIS marine data: Available

Satellite SST: Unavailable

Recommendation confidence has been reduced.

Do not hide the limitation.

Do not invent missing data.

36. **needs_input** State

Backend may return:

```
{
  "status": "needs_input",
  "questions": [
    {
      "id": "location",
      "type": "location",
      "question": "What fishing location should I use?"
    }
  ]
}
```

```
}  
]  
}
```

Frontend should render an interactive clarification.

Example:

I can help with that.

Which fishing location should I use?

[Use my location]

[Select on map]

[Enter location]

37. Multilingual UI

Backend supports language detection.

At minimum support:

English

Hindi

Malayalam

Tamil

Urdu

The frontend should support language switching.

Language should not be determined only by browser language.

The backend response contains the authoritative response language.

38. Indian Regional Language UX

Ensure fonts, text wrapping, labels and charts support Indic scripts correctly.

Test:

English

हिन्दी

മലയാളം

தமிழ்
اردو

Do not hard-code English-only UI assumptions.

39. Voice Interface

Voice can be implemented as an input method.

Flow:

```
graph TD
    A[User speaks] --> B[Speech-to-text]
    B --> C[Natural-language query]
    C --> D[/api/v1/orchestrate/]
    D --> E[Response]
    E --> F[Optional text-to-speech]
```

Voice processing must not bypass the backend orchestration pipeline.

40. Location

Support:

- GPS
- Map selection
- Manual coordinates
- Place search

Location object:

```
{
  "latitude": 9.9312,
  "longitude": 76.2673,
  "accuracy": 12,
  "timestamp": "...",
  "source": "GPS"
}
```

}

41. Units

Support user preferences for:

- km / nautical miles
- metres / feet where applicable
- °C / °F
- km/h / knots

However, backend-provided values must remain authoritative.

Unit conversion is presentation-only.

42. API Layer

Create a dedicated API client.

Example:

lib/api/

orchestrate.ts

alerts.ts

health.ts

Do not call `fetch()` throughout components.

Instead:

Component

↓

Hook

↓

API client

↓

Backend

43. API Configuration

Use environment variables.

Example:

```
NEXT_PUBLIC_API_BASE_URL  
NEXT_PUBLIC_WS_BASE_URL
```

Never hard-code production backend URLs.

Never put secrets in `NEXT_PUBLIC_*`.

44. Authentication

If authentication is required by the backend, centralize it in the API client.

Do not implement authentication separately in every component.

45. Error Handling

Every API call must handle:

```
400  
401  
403  
404  
408  
409  
429  
500  
502  
503  
504
```

Map backend error codes to user-friendly messages.

Example:

```
SOURCE_UNAVAILABLE
```

Some marine data is temporarily unavailable.
The response may be incomplete.

Do not display raw exceptions.

46. Loading States

Do not use blank screens.

Use meaningful loading states.

Example:

Analyzing marine conditions...

- ✓ Location identified
 - ✓ Checking marine data
 - ✓ Checking weather
 - Checking restrictions
 - Assessing risk
-

47. Empty States

Examples:

No active alerts in this area.

No PFZ information is currently available.

No route could be generated for the selected locations.

48. Accessibility

The frontend must support:

- keyboard navigation
- screen readers
- sufficient contrast
- visible focus states
- semantic HTML
- accessible map controls
- accessible alert notifications

Do not communicate critical safety information using color alone.

Example:

⚠ HIGH RISK

rather than only a red background.

49. Responsive Design

Desktop:

Chat | Map

Tablet:

Chat

Map

Mobile:

Header

Chat

Bottom navigation

The map must remain usable on mobile.

50. Performance

Avoid rendering thousands of geospatial features directly as React components.

Use map-native layers.

Prefer:

GeoJSON source

+

Map layer

instead of:

10,000 React markers

Use clustering where appropriate.

Lazy-load heavy map/chart components.

51. Security

Never expose:

DATABASE_URL
OPENAI_API_KEY
MCP credentials
source API credentials

in the frontend.

Never place secrets into:

NEXT_PUBLIC_*

Validate user input.

Sanitize rendered content.

Do not execute arbitrary HTML from backend responses.

52. Prompt Injection Protection

The frontend must treat backend responses as untrusted data.

Do not execute instructions contained inside:

- marine documents
- retrieved RAG content
- source advisories
- external text
- tool responses

Only render them as content.

53. No Direct MCP

The frontend must NEVER do:

Frontend



MCP

Correct:

Frontend



FastAPI



Orchestrator



MCP

54. No Direct Database

Never implement:

Frontend



Supabase/PostgreSQL

Correct:

Frontend



API



Backend



PostgreSQL/PostGIS/pgvector

55. No Agent-Specific UI Coupling

Do not create:

WeatherAgentPage

MarineAgentPage

MCPToolPage

RiskEnginePage

The frontend should represent user-facing capabilities.

Use:

Marine Conditions

Safety

Fishing

Route

Scenario

Knowledge

56. Recommended Component Architecture

Create reusable components:

ChatInterface

ChatMessage

UserMessage

AssistantMessage

StreamingMessage

ClarificationCard

EvidencePanel

ConfidenceBadge

RiskBadge

DataFreshnessBadge

MarineMap

MapControls

MapLayerControl

MapLegend

MapFeaturePopup

RestrictionOverlay

RouteOverlay

PFZOverlay

WeatherCard

MarineConditionsCard

PFZCard

ProductivityCard

TimeSeriesChart

ChartToolbar

AlertPanel

AlertCard

AlertCenter

RoutePanel

RouteSummary

RouteAlternative

SourcePanel

ProvenancePanel

57. Response Renderer

Do not render every answer as plain text.

Build a response renderer that can interpret:

answer

risk

evidence

map

charts

alerts

route

limitations

Example:

Assistant Response

Answer	
--------	--

Risk	
------	--

Map	
-----	--

Evidence	
----------	--

Charts	
--------	--

58. Backend Response Contract

The frontend should expect a canonical structure similar to:

```
{
  "request_id": "req_123",
  "run_id": "run_456",
  "status": "completed",
  "language": "en",

  "answer": "...",

  "confidence": {
    "score": 0.91,
    "level": "high"
  },

  "risk": {
    "classification": "CAUTION",
    "reason": "..."
  },

  "evidence": [],

  "provenance": [],

  "limitations": [],

  "map": {
    "features": []
  },

  "charts": [],

  "alerts": [],

  "route": null
}
```

Use the actual backend OpenAPI specification as the source of truth when available.

Do not invent incompatible fields.

59. Map Contract

Expected structure:

```
{
  "map": {
    "features": [
      {
        "id": "feature-1",
        "type": "Feature",
        "geometry": {},
        "properties": {
          "layer": "restriction",
          "severity": "critical",
          "source": "NHO",
          "valid_from": "...",
          "valid_to": "..."
        }
      }
    ]
  }
}
```

Render the geometry directly.

60. Chart Contract

Conceptually:

```
{
  "id": "sst",
  "name": "Sea Surface Temperature",
  "unit": "°C",
  "source": "ISRO",
  "timestamps": [],
  "values": []
}
```

61. Alert Contract

Conceptually:

```
{
  "id": "alert-123",
  "type": "WAVE",
  "severity": "HIGH",
  "title": "High wave conditions",
  "message": "...",
  "location": {},
  "geometry": {},
  "valid_from": "...",
  "valid_to": "...",
  "source": "INCOIS"
}
```

62. Route Contract

Conceptually:

```
{
  "route": {
    "status": "selected",
    "selected_route": {
      "geometry": {},
      "distance_km": 42.3,
      "risk": "low"
    },
    "alternatives": []
  }
}
```

63. Evidence Contract

Conceptually:

```
{
  "id": "evidence-1",
  "type": "FORECAST",
  "source": "INCOIS",
  "source_reference": "...",
  "timestamp": "...",
  "validity": {},
  "claim": "Wave height",
  "value": 2.1,
  "unit": "m",
}
```

```
"confidence": 0.92
}
```

64. Frontend Data Flow

Implement:

User submits query



Create/reuse session_id



POST /api/v1/orchestrate



Receive run_id



Subscribe to stream if supported



Display progress



Receive final response



Render answer



Render risk



Render map



Render charts



Render alerts



Render evidence

65. Conversation History

Each conversation should store:

session_id

messages

timestamp

title

last_updated

Do not store sensitive backend execution internals.

66. Suggested Homepage

The first screen should immediately communicate the purpose of FloatChat.

Example:

FLOATCHAT

Marine Intelligence, Simplified.

Ask anything about the ocean, weather, fishing,
safety, routes and marine conditions.

Is it safe to fish tomorrow morning near Kochi?

>

Popular questions

[Nearest PFZ] [Marine Conditions] [Safe Route]
[Weather] [Restrictions] [Fishing Zones]

Avoid excessive dashboard widgets on initial load.

The conversational experience should remain the primary interaction.

67. Marine Intelligence Dashboard

After a query, the interface can expose contextual cards:

CURRENT CONDITIONS

SST	28.4°C
Chlorophyll	High
Wave Height	1.4 m
Wind	16 km/h
Risk	CAUTION

These values must come from backend responses.

68. Scenario / What-If UI

Support questions such as:

What if I leave at 5 AM instead?

or:

What if I use the southern route?

The frontend should provide an optional scenario interaction:

Scenario

Departure:

[05:00]

Route:

[Current route]

[Evaluate Scenario]

The actual scenario reasoning happens entirely in the backend.

69. Knowledge / RAG UI

Knowledge responses may contain:

- regulations
- marine advisories
- research
- reports
- historical documents

Display citations/evidence.

Example:

Sources

INCOIS Advisory

Marine Advisory Bulletin

2026-08-31

Research Paper

Seasonal variability of chlorophyll...

Do not expose vector-search implementation.

Do not say:

pgvector similarity = 0.83

unless specifically useful and approved for user-facing explainability.

70. Search

If a location/search box exists, support:

Place

Port

Coastal region

Coordinates

The backend remains authoritative for marine/geospatial interpretation.

71. Map ↔ Chat Integration

This is important.

If the user clicks a map location:

Map

↓

Selected coordinates

↓

Chat context

Then the input can show:

Location selected:

9.9312° N, 76.2673° E

User:

What are the conditions here?

Frontend sends the selected location with the request.

72. Chat ↔ Map Integration

If the backend returns PFZ or route information:

Assistant response



Map automatically updates

Example:

Nearest PFZ found.

[View on Map]

Clicking it centers the map.

73. Chat ↔ Alert Integration

If an answer contains an active hazard:

⚠ High Wave Alert

Clicking the alert should:

open alert

+

zoom map

+

show evidence

74. Visual Design Direction

The product should feel:

Marine

Technical

Trustworthy
Calm
Operational
Professional

Avoid making it look like:

- a generic chatbot
- a gaming dashboard
- a stock-trading terminal
- an overly futuristic AI demo

The system is intended for real operational decision support.

75. Color Semantics

Use consistent semantic states.

For example:

SAFE → positive
CAUTION → warning
HIGH_RISK → danger
CRITICAL → critical
RESTRICTED → restriction
UNKNOWN → neutral

Do not rely exclusively on colors.

Always combine:

icon + label + color

76. Typography

Use a highly readable font.

Prioritize:

- numeric readability
- Indic language support
- clear hierarchy
- dense but readable marine information

Avoid extremely thin font weights.

77. Desktop Map Controls

Provide:

+ / -
My Location
Layers
Legend
Search
Fullscreen
Measure

Do not overload the map with unnecessary controls.

78. Map Legend

Always show a legend when multiple layers are active.

Example:

LEGEND

- Vessel
 - PFZ
 - ▲ Hazard
 - Route
 - ▨ Restricted Area
-

79. Notifications

Use non-intrusive notifications for ordinary events.

Critical safety events may require prominent banners.

Do not repeatedly notify users for the same unchanged alert.

80. Offline / Cached UI

The frontend may cache previous responses for UX.

However:

Cached data must never be presented as live data.

Clearly display:

Cached response

Last updated: ...

Never imply stale information is current.

81. Real-Time Data Principle

The backend determines:

freshness

validity

source availability

confidence

risk

Frontend determines only:

presentation

interaction

navigation

visualization

82. Testing

Implement:

Unit tests

Test:

- API client
- response parsing

- risk display
- map transformation
- chart transformation
- error mapping
- localization
- formatting

Component tests

Test:

- chat
- map
- alerts
- evidence
- route
- charts

Integration tests

Test:

query

→ API

→ response

→ UI

using mocked backend responses.

83. Required Frontend Acceptance Scenarios

Implement and test at minimum:

1. PFZ

Where is the nearest PFZ today?

Expected:

- answer
- PFZ map
- evidence
- confidence

2. Safety

Is it safe to fish tomorrow morning?

Expected:

- risk classification
- reasoning
- evidence
- relevant hazards

3. Weather

What is the weather and sea condition near my location?

Expected:

- weather
- waves
- charts
- map

4. Restriction

Are there any restricted areas nearby?

Expected:

- restriction layer
- warning
- evidence

5. Route

Find the safest route.

Expected:

- route
- alternatives
- risk
- restrictions

6. Scenario

What if I leave two hours earlier?

Expected:

- scenario comparison
- updated recommendation

7. Knowledge

Why has productivity declined?

Expected:

- explanation
- evidence
- source references

8. Multilingual

Test:

Malayalam

Hindi

Tamil

Urdu

9. Multi-turn

Test contextual follow-ups.

10. Degraded source

Simulate unavailable marine/weather source.

The UI must remain functional.

84. Do Not Fake Production Data

During development, mock data may be used.

However:

MOCK DATA

must be clearly separated from:

LIVE BACKEND DATA

Never silently fall back from the production API to fabricated marine information.

85. Environment Configuration

Provide:

`.env.example`

containing only non-secret configuration placeholders.

Example:

```
NEXT_PUBLIC_API_BASE_URL=  
NEXT_PUBLIC_WS_BASE_URL=  
NEXT_PUBLIC_MAP_STYLE_URL=
```

Never commit:

`.env`

with secrets.

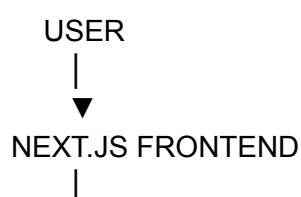
86. Documentation Required

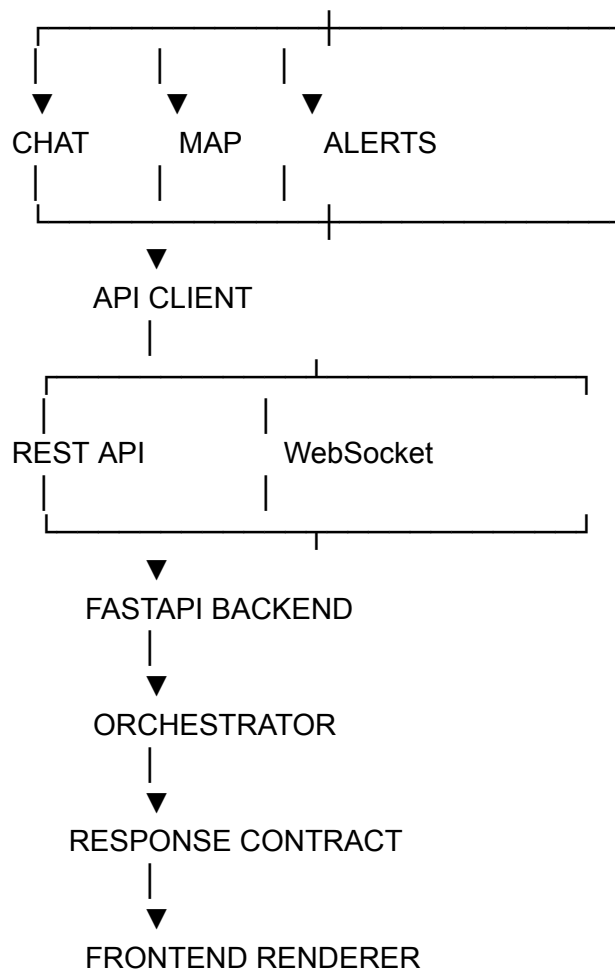
Create:

```
docs/  
├── architecture.md  
├── api-integration.md  
├── map.md  
├── streaming.md  
├── state-management.md  
├── testing.md  
└── deployment.md
```

87. Frontend Architecture Diagram

Document this architecture:





88. Strict Separation of Responsibilities

Frontend owns

- UX
- UI
- navigation
- interaction
- map rendering
- chart rendering
- animation
- responsive design
- accessibility
- local formatting
- session ID handling
- API communication
- stream rendering

Backend owns

intent
planning
agent selection
MCP
data retrieval
data fusion
spatial reasoning
temporal reasoning
ML
RAG
risk
safety
verification
confidence
provenance
source freshness
restrictions
route computation

89. Important Safety Rule

The frontend must never override backend safety decisions.

Never implement:

if wave_height < 2:
SAFE

Never implement:

if no alert:
SAFE

Never implement:

if restriction not displayed:
SAFE

The frontend only displays:

backend.risk.classification

90. Important Data Integrity Rule

Never fabricate:

- SST
- chlorophyll
- wave height
- weather
- current
- PFZ
- restriction
- route
- risk
- confidence

If the backend does not provide a value:

N/A

or:

Unavailable

must be displayed.

91. Important Explainability Rule

The frontend must make it easy to answer:

"Why did FloatChat say this?"

Every major recommendation should have:

Recommendation

+

Risk

+

Evidence

+

Source

+

Freshness

+

Confidence

92. Important Operational Rule

This is a decision-support platform.

Do not design the UI as though the AI is infallible.

Include appropriate language such as:

Based on available marine and weather data.

and:

Conditions may change. Check current official advisories before departure.

The backend's evidence and limitations should drive this messaging.

93. Final Integration Contract

The frontend developer should be able to build the application using only:

1. Backend OpenAPI specification
2. API integration document
3. Response schemas
4. Example requests
5. Example responses
6. Streaming event specification

They should **not need to inspect**:

orchestrator.py

planner.py

executor.py

agents.py

MCP tools

database schema

PostGIS implementation

pgvector implementation

ML models

risk engine implementation

94. Final Development Sequence

Implement in this order:

PHASE A

Project setup



API client



Type generation



Base layout

PHASE B

Chat



Session handling



Orchestration API



Response rendering

PHASE C

Streaming



Progress UI



Error handling

PHASE D

Marine map



GeoJSON



Layer system



PFZ



Restrictions



Hazards



Routes

PHASE E

Charts



Marine conditions



Weather

↓
Productivity

PHASE F
Alerts

↓
Geofencing UI

↓
Notifications

PHASE G
Evidence

↓
Provenance

↓
Confidence

↓
Explainability

PHASE H
Multilingual

↓
Voice

↓
Responsive UX

↓
Accessibility

PHASE I
Testing

↓
Performance

↓
Security

↓
Production deployment

95. Definition of Done

The frontend is complete only when:

- Next.js/React/TypeScript application is production-ready.
- Frontend is in a separate repository.
- Backend URL is environment-configurable.
- OpenAPI-generated types/client are used where practical.

- Chat works against `/api/v1/orchestrate`.
 - Sessions work.
 - Multi-turn conversations work.
 - Streaming works.
 - `needs_input` works.
 - Error states work.
 - Degraded states work.
 - Multilingual responses work.
 - Marine map works.
 - GeoJSON rendering works.
 - PFZ visualization works.
 - SST visualization works.
 - Chlorophyll visualization works.
 - Weather visualization works.
 - Wave visualization works.
 - Hazard visualization works.
 - Restriction visualization works.
 - Geofence notifications work.
 - Route visualization works.
 - Route alternatives work.
 - Charts work.
 - Alerts work.
 - Evidence panel works.
 - Provenance works.
 - Confidence display works.
 - Risk classification works.
 - No frontend safety logic overrides backend.
 - No direct database access exists.
 - No direct MCP access exists.
 - No source API credentials exist in frontend.
 - No agent-specific coupling exists.
 - No fabricated production marine data exists.
 - Cached data is clearly labelled.
 - Accessibility requirements are addressed.
 - Responsive layouts work.
 - API failures are handled gracefully.
 - Security checks pass.
 - Component tests pass.
 - Integration tests pass.
 - Production build succeeds.
 - Documentation is complete.
-

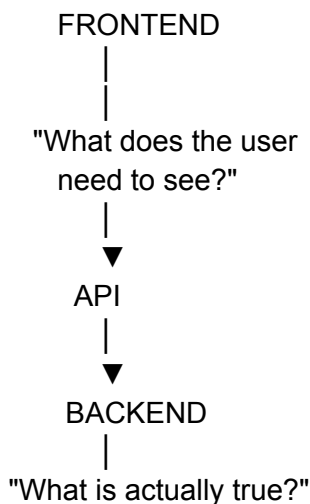
96. Final Instruction to the Frontend Development Team

Build FloatChat as a **serious marine decision-support interface**, not as a generic AI chatbot.

The primary user experience should be:

ASK
↓
UNDERSTAND
↓
SHOW PROGRESS
↓
ANSWER
↓
VISUALIZE
↓
EXPLAIN
↓
PROVIDE EVIDENCE

The most important principle is:



The frontend **renders the backend's authoritative decision-support output**. It must never independently determine marine safety, restrictions, risk, route legality, data validity, or confidence.

The final product should make a fisherman, maritime operator, researcher, or coastal authority feel that they can ask a complex marine question naturally and immediately receive a **clear answer + map + relevant data + risk + evidence + provenance**, without needing to understand the complex Agentic AI infrastructure operating behind the interface.

